

Predictive Maintenance Analysis

Presentation Agenda

01

Introduction & Dataset

Project goals, predictive maintenance basics, and raw data overview.

02

Exploratory Data Analysis

Visualizing correlations, sensor distributions, and failure modes.

03

Data Preparation & Modeling

Feature engineering, SMOTE balancing, and model selection.

04

Results & Business Impact

Performance metrics, comparison, and practical applications.

Project Team

❖ Group (3)

Mohamed Elmekawy	Introduction & Dataset
Mahmoud Elkafay	Exploratory Data Analysis
Talaat Sallam	Data Preparation & Modeling
Eman Gabr	Results & Business Impact

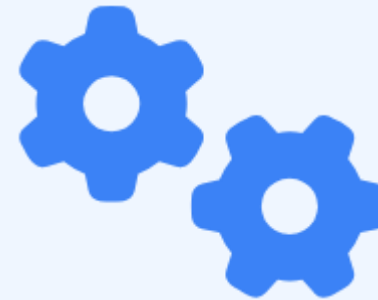
Section 1: Introduction & Dataset



Predictive Maintenance (PdM)

Predictive maintenance uses data-driven, proactive methods to evaluate the condition of equipment and predict when maintenance should be performed.

- Reduces unplanned downtime
- Optimizes maintenance schedules
- Maximizes tool life and resource usage
- Improves overall operational safety



Project Problem Statement

Industrial machines often fail unexpectedly, leading to significant economic losses. The challenge is to distinguish between normal operation and impending failure using real-time sensor data.

Objectives:

- Analyze the AI4I 2020 synthetic dataset for predictive patterns.
- Address the severe class imbalance (only 3.39% failure rate).
- Develop an ML model to predict failure with high recall to avoid missing critical issues.

Dataset Overview

The AI4I 2020 Predictive Maintenance Dataset contains 10,000 observations representing industrial processes.

Attribute	Value
Total Samples	10,000
Total Features	14 Columns
Failure Samples	339 (3.39%)
Non-Failure Samples	9,661 (96.61%)
Data Integrity	0 Missing Values

Predictive Maintenance Parameters

Input Sensor Features

Thermal

- Air Temperature [K]
- Process Temperature [K]

Mechanical

- Rotational Speed [rpm]
- Torque [Nm]

Structural & Categorical

- Tool Wear: Usage time [min]

- Type: Quality Level (L, M, H)

! Target Variable: Failure

Normal (0)

9661

Failure (1)

339

28.5:1

IMBALANCE RATIO

3.39%

FAILURE RATE

Section 2: Exploratory Data Analysis

1- Dataset Overview

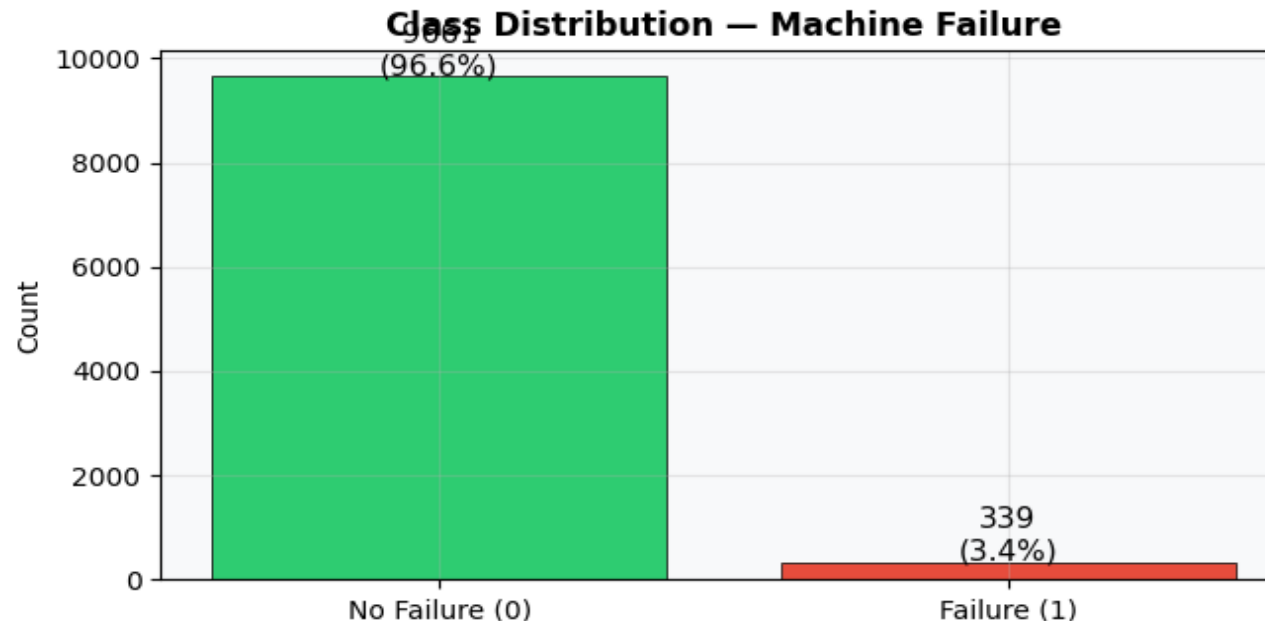
ML models cannot learn properly if:

- The dataset is highly imbalanced (9661 normal vs 339 failures).
- Data is small

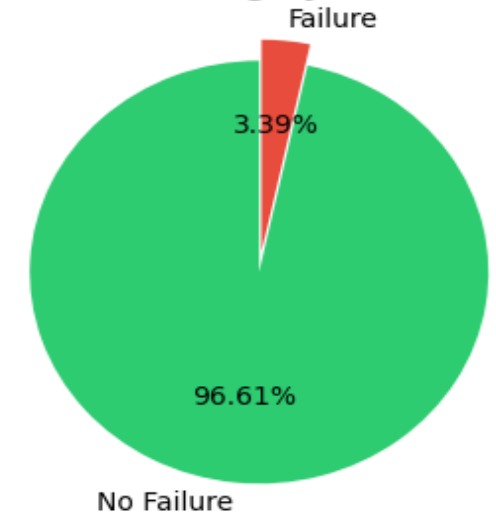
Observation: Class imbalance ratio is **28.5:1**.

So we use SMOTE stands for: **Synthetic Minority Over-sampling Technique**

Target Variable Analysis



Failure Rate (Highly Imbalanced)



2- Checking Missing Values and Data Types

ML models cannot learn properly if:

- many values are missing
- features are corrupted

```
=== Statistical Summary ===
UDI                | dtype: int64      | unique: 10000
Product ID         | dtype: object     | unique: 10000
Type               | dtype: object     | unique: 3
Air temperature [K] | dtype: float64    | unique: 93
Process temperature [K] | dtype: float64    | unique: 82
Rotational speed [rpm] | dtype: int64      | unique: 941
Torque [Nm]        | dtype: float64    | unique: 577
Tool wear [min]    | dtype: int64      | unique: 246
Machine failure    | dtype: int64      | unique: 2
TWF                | dtype: int64      | unique: 2
HDF                | dtype: int64      | unique: 2
PWF                | dtype: int64      | unique: 2
OSF                | dtype: int64      | unique: 2
RNF                | dtype: int64      | unique: 2
```

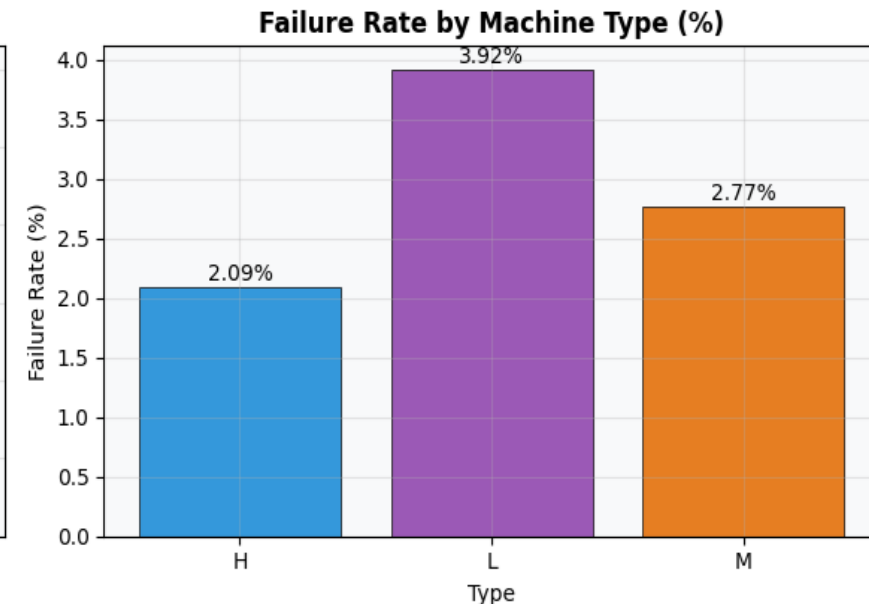
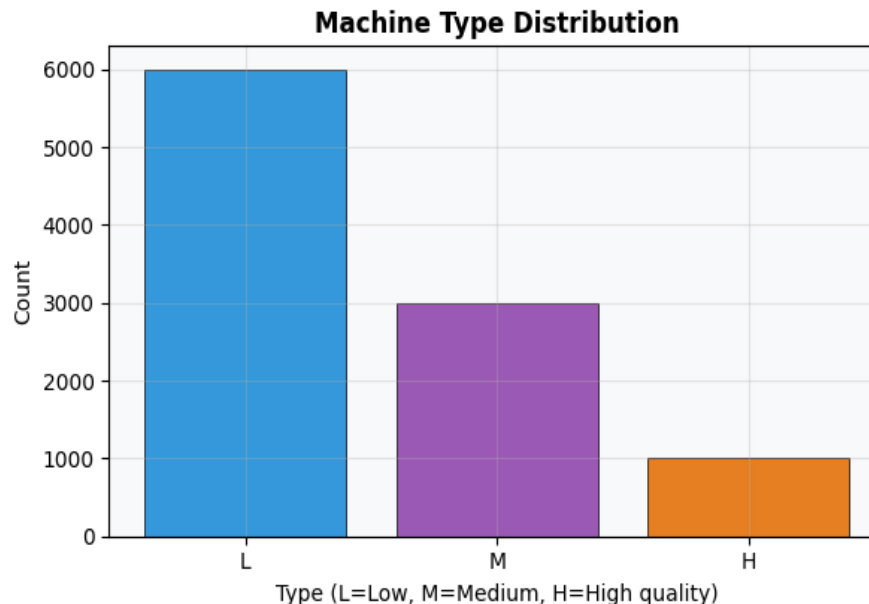
```
=== Missing Values ===
UDI                0
Product ID        0
Type              0
Air temperature [K] 0
Process temperature [K] 0
Rotational speed [rpm] 0
Torque [Nm]       0
Tool wear [min]   0
Machine failure    0
TWF               0
HDF               0
PWF               0
OSF               0
RNF               0
dtype: int64
```

3- Machine Type Analysis

Distribution of Machine Types (L, M, H) and their associated failure rates:

Quality Type	Count	Failures	Failure Rate
Low (L)	6000	235	3.92%
Medium (M)	2997	83	2.77%
High (H)	1003	21	2.09%

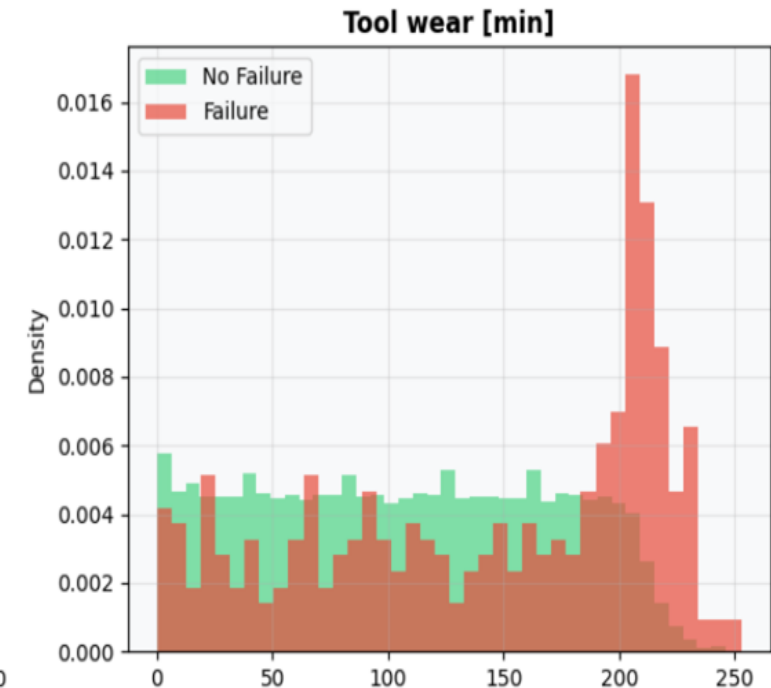
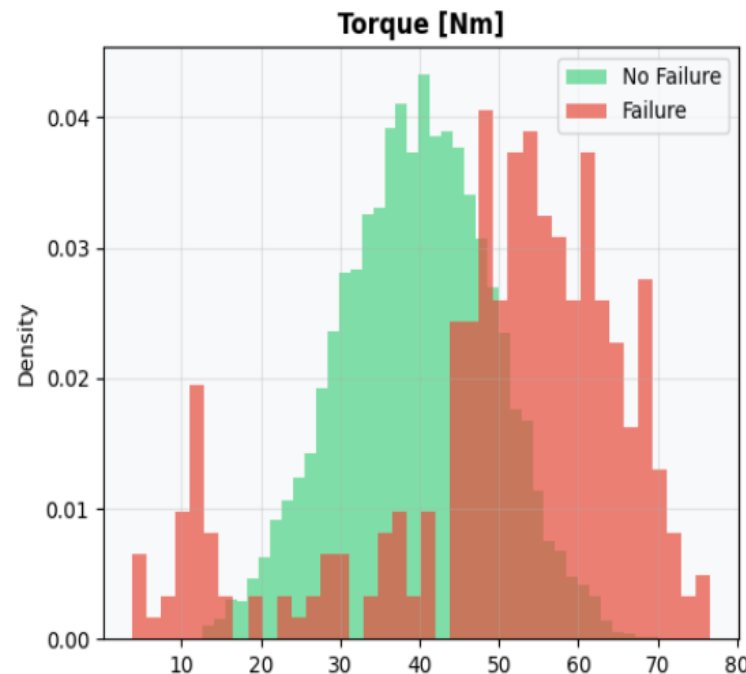
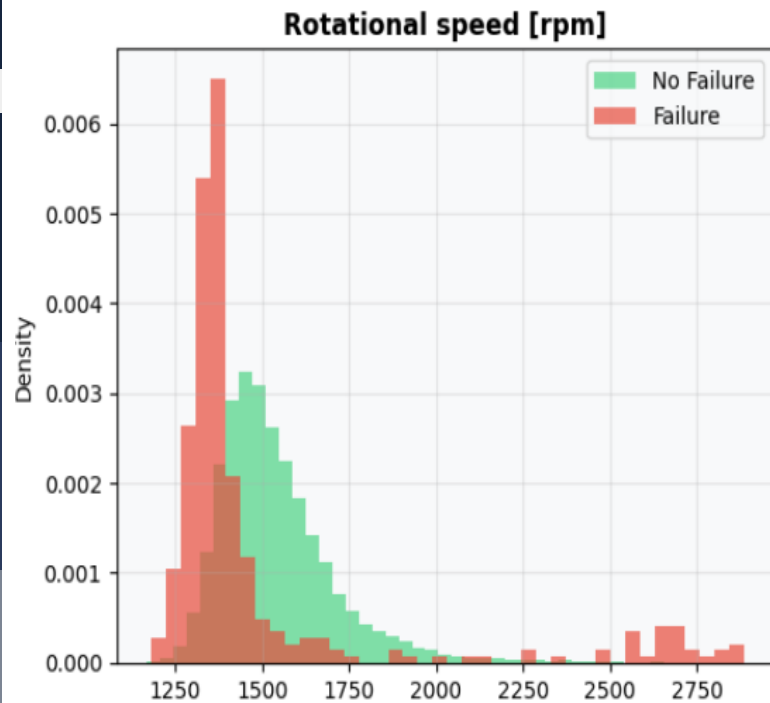
Observation: Low-quality machine types (L) exhibit nearly double the failure rate of High-quality (H) machines, suggesting quality level is a significant predictor.



4- Sensor Reading Distributions

Analyzing how sensor values differ between Failure and No Failure states:

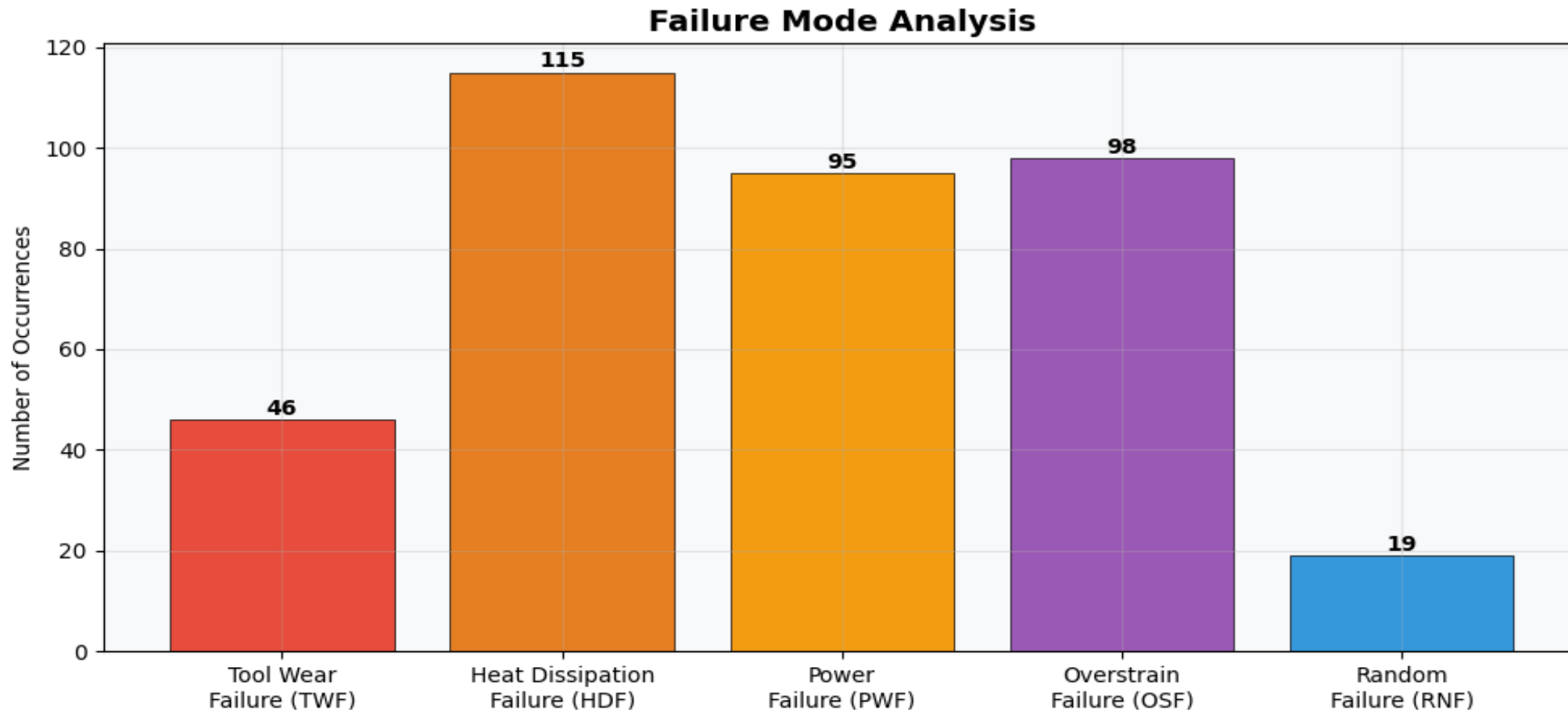
- **Tool Wear:** Failures are more common as tool wear approaches 200 minutes.
- **Torque:** Higher torque values (>60 Nm) correlate strongly with failures.
- **Rotational Speed:** Lower speeds generally associated with higher failure frequency.



5- Analysis of Failure Modes

Five specific failure types were recorded in the dataset:

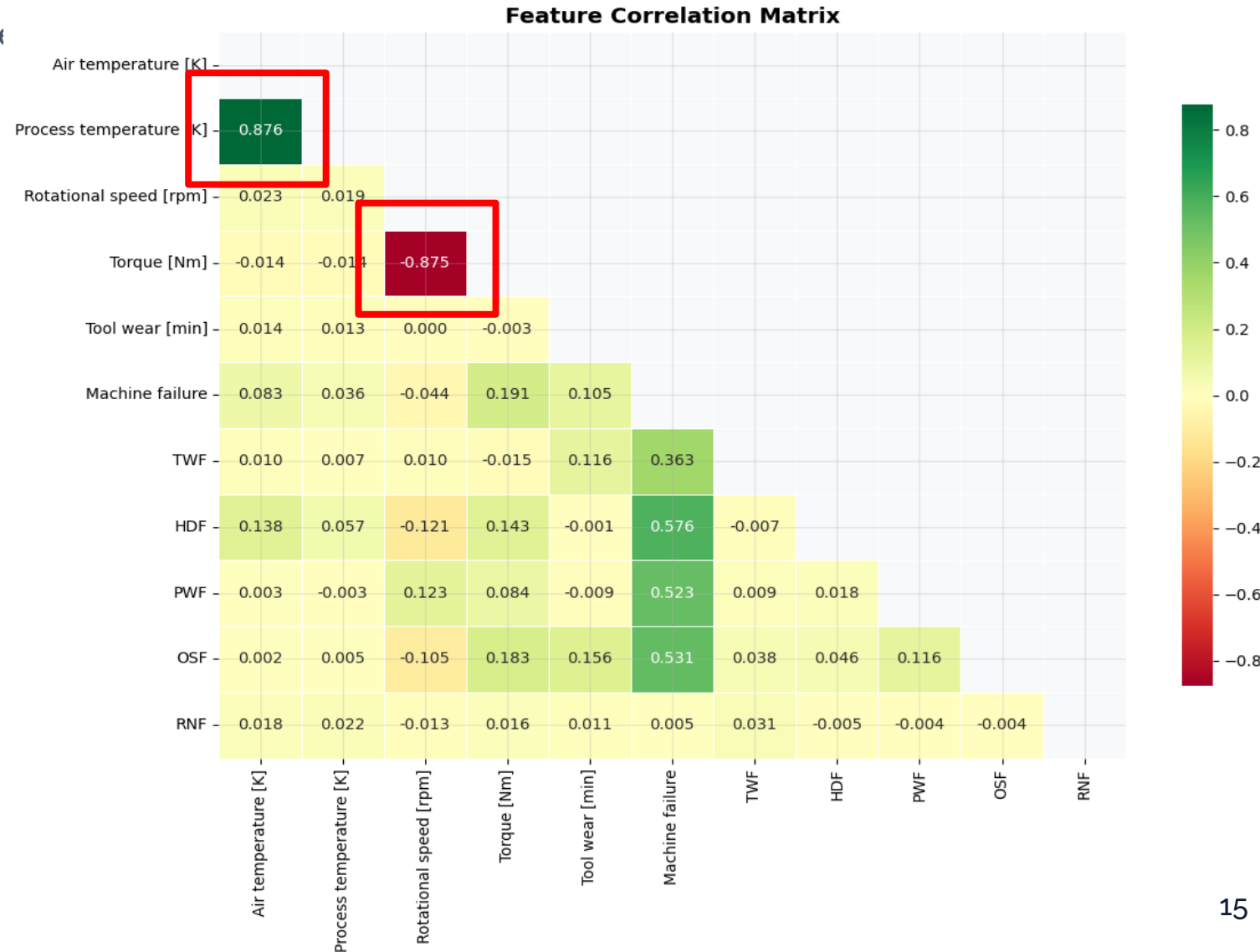
Observation: Most failures arise from **thermal and mechanical issues** rather than random events, indicating that sensor measurements can effectively predict failures.



6- Correlation Matrix Key Findings

Numerical correlations revealed critical interdependencies:

- **Strong Negative Correlation:** Torque and Rotational Speed (~ -0.875).
- **Strong Positive Correlation:** Air and Process temperatures (~ 0.876).
- **Target Correlation:** Torque and Tool Wear show the strongest individual linear relationships with Machine Failure.



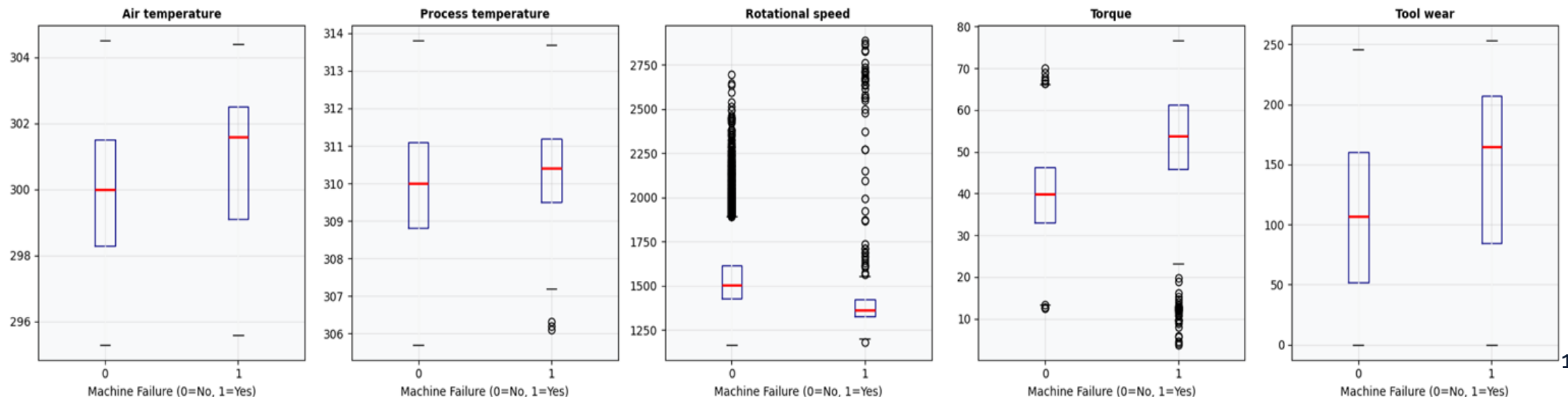
7- Outlier & Variance Analysis

Boxplots show that **failure machines** have **more variation** and **more unusual sensor values** than normal machines

Key Insight: In predictive maintenance, unusual sensor readings may indicate developing faults, making outliers valuable for failure prediction rather than noise to be removed.

- Failure machines tend to operate at lower rotational speeds and higher torque values.
- Tool wear is significantly higher in failure cases.
- Several outliers are observed, particularly in rotational speed and torque.
- These outliers were retained because they represent valid abnormal operating conditions rather than data errors.

Sensor Readings — Boxplot by Failure Label



Key EDA Insights

Quality Matters

Machine Type 'L' is more failure-prone than 'H'.

The Speed-Torque Tradeoff

High torque at low speed is a primary failure indicator.

Multi-Failure

Some failures involve multiple modes simultaneously.



Section 3: Data Preparation & Modeling

Feature Engineering Strategy

To improve model sensitivity, we created physical-based features derived from raw sensor readings:

- **Thermal Dynamics:** Temperature difference between process and air.
- **Physics-Based:** Calculated mechanical power and torque-speed ratios.
- **Wear Dynamics:** Wear rate relative to rotational speed.

New Engineered Features

Feature Name	Derivation Logic
temp_diff	Process Temp - Air Temp
power_W	$\text{Torque} \times \text{Rotational Speed} \times (2\pi/60)$
wear_rate	Tool Wear / Rotational Speed
heat_stress	$\text{Process Temp} \times \text{Tool Wear} / 1000$

Preprocessing Pipeline

1. Encoding

Converting 'Type' to numeric
(L:1, M:2, H:0).

2. Balancing

Applying SMOTE to generate
synthetic failure samples.

3. Scaling

Standardizing all features to
Mean 0, Std 1.

Handling Class Imbalance

Synthetic Minority Over-sampling Technique (SMOTE):

Without balancing, models tend to predict "No Failure" for everything, yielding high accuracy but 0% recall for failures.

- **Before:** 9,661 (Non-Fail) vs 339 (Fail)
- **After SMOTE:** 9,661 (Non-Fail) vs 9,661 (Fail)
- **Result:** Balanced dataset with 19,322 total samples.

Standardization (Scaling)

Sensor values have vastly different ranges (e.g., Speed ~1500 vs Torque ~40). We use **StandardScaler**:

$$z = (x - u) / s$$

This is crucial for distance-based models (KNN, SVM) and gradient-descent based optimization.

Verification: After scaling, the training set mean is ~0.0000 for all features, ensuring numerical stability.

Data Splitting Strategy

Split applied to the resampled (SMOTE) balanced data:

Train (70%)

13,532 samples used for
model learning.

Val (15%)

2,891 samples for
hyperparameter tuning.

Test (15%)

2,899 samples for final
evaluation.

Machine Learning Candidates

Seven different algorithms were evaluated to find the optimal PdM solution:

- **Linear:** Logistic Regression, SVM (RBF Kernel).
- **Trees:** Decision Tree, Random Forest.
- **Ensemble:** Gradient Boosting, AdaBoost.
- **Instance-Based:** K-Nearest Neighbors (KNN).



Section 4: Results & Business Impact

Logistic Regression & Decision Tree

Logistic Regression

- F1-Score: 0.8711
- AUC-ROC: 0.9409
- Simple but lacks non-linear capture.

Decision Tree

- F1-Score: 0.9657
- AUC-ROC: 0.9655
- High variance; prone to overfitting.

KNN & SVM Performance

Model	Recall	F1-Score	AUC-ROC
K-Nearest Neighbors	0.9855	0.9523	0.9842
SVM (RBF)	0.9586	0.9407	0.9840

Note: KNN shows exceptional recall (98.5%), meaning it rarely misses a failure, though it is computationally more expensive.

All Models Comparison

=== BASELINE RESULTS ===

	Accuracy	Precision	Recall	F1-Score	AUC-ROC	MCC
Model						
Random Forest	0.9796	0.9734	0.9862	0.9798	0.9982	0.9594
Gradient Boosting	0.9517	0.9468	0.9572	0.9520	0.9911	0.9035
K-Nearest Neighbors	0.9507	0.9213	0.9855	0.9523	0.9842	0.9035
SVM (RBF)	0.9396	0.9235	0.9586	0.9407	0.9840	0.8799
AdaBoost	0.9224	0.9310	0.9124	0.9216	0.9771	0.8449
Decision Tree	0.9655	0.9610	0.9703	0.9657	0.9655	0.9311
Logistic Regression	0.8713	0.8726	0.8696	0.8711	0.9409	0.7427



Top Performer: Random Forest

The **Random Forest Classifier** achieved the best overall results across all metrics:

- **Accuracy:** 97.96%
- **Recall:** 98.62%
- **F1-Score:** 0.9798
- **AUC-ROC:** 0.9982

Why it won: By combining 100 decorrelated decision trees, it effectively captured the complex thermal-mechanical interactions while maintaining high robustness against noise.

Final Model Comparison

Sorted by AUC-ROC Score (Highest to Lowest):

Model	F1-Score	AUC-ROC	MCC
Random Forest	0.9798	0.9982	0.9594
Gradient Boosting	0.9520	0.9911	0.9035
K-Neighbors	0.9523	0.9842	0.9035
SVM (RBF)	0.9407	0.9840	0.8799
AdaBoost	0.9216	0.9771	0.8449

Feature Engineering Evaluation

Original Features Only

```
original_feature_cols = [  
    'Type_enc',  
    'Air temperature [K]',  
    'Process temperature [K]',  
    'Rotational speed [rpm]',  
    'Torque [Nm]',  
    'Tool wear [min]'  
]
```



Original + Engineered Features

```
#Prepare feature matrix and target  
feature_cols = [  
    'Type_enc',  
    'Air temperature [K]',  
    'Process temperature [K]',  
    'Rotational speed [rpm]',  
    'Torque [Nm]',  
    'Tool wear [min]',  
    'temp_diff',  
    'power_W',  
    'wear_rate',  
    'torque_speed_ratio',  
    'heat_stress'  
]
```


Feature Engineering Evaluation

The Random Forest model was trained twice to evaluate the impact of feature engineering. The engineered features were designed to capture machine stress, power consumption, and wear behavior.

=== RANDOM FOREST COMPARISON ===

	Without FE	With FE
Accuracy	0.9748	0.9796
Precision	0.9611	0.9734
Recall	0.9896	0.9862
F1-Score	0.9752	0.9798
AUC-ROC	0.9977	0.9982
MCC	0.9501	0.9594

Feature Engineering Evaluation

Impact of Feature Engineering on Random Forest

Positive findings:

Accuracy improved by **0.49%**

Precision improved by **1.28%**

F1-score improved by **0.47%**

MCC improved by **0.98%**

AUC-ROC improved by **0.05%**

Feature engineering provided a measurable improvement in Random Forest performance. The increase in Precision, F1-Score, MCC, and AUC-ROC indicates that the engineered features contributed additional information related to machine operating conditions and failure behavior.

Business & Operational Impact

Cost Savings

Predicting failures before they happen prevents expensive collateral damage to machine components.

Production Continuity

Reducing downtime by scheduling repairs during planned breaks rather than emergency stops.

Resource Efficiency

Tools are replaced only when the model predicts failure, not on fixed (conservative) intervals.

Any Questions ?

Thank you for your attention.